AH_G474 User Manual

Getting started, hardware guide and golden tips for the STM32G474RBT6 header board: power, clocks, boot, programming, USB, I/O and every major peripheral, matched to the way the board is wired

5 steps	170 MHz	50 mA	SWD + USB
from the USB cable to a blinking LED	PLL recipe for the 16 MHz crystal	load limit of the 3.3 V regulator	two ways to program and debug

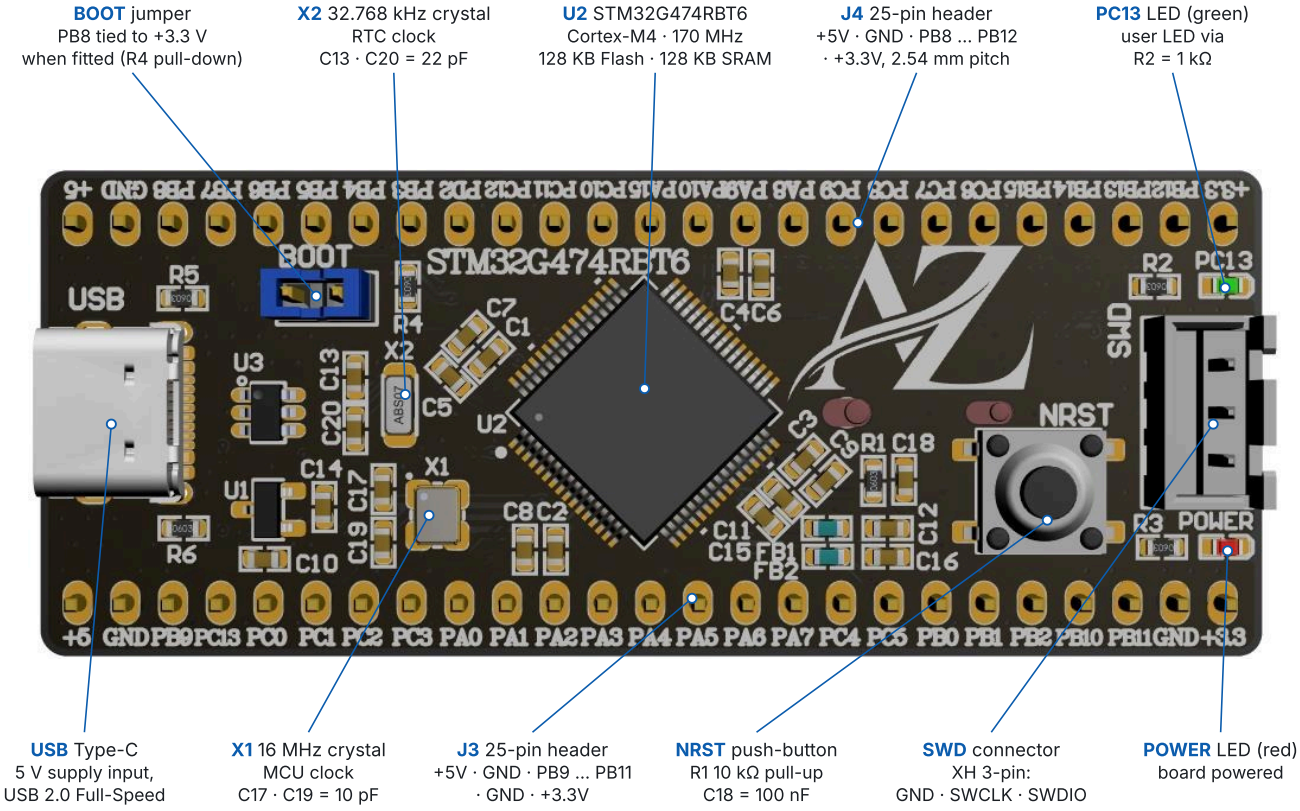


Figure 1. The AH_G474 board (top view) with the controls and connectors used in this manual. Component reference designators match the silkscreen and the schematic.

Contents

1. Introduction 2
2. Safety and Limits 2
3. Quick Start 3
4. Board Tour 3
5. Power 4
6. Clocks 5
7. Reset, Boot and Programming 6
8. USB Type-C 7
9. I/O Headers 8
10. Peripheral Guide 10
    - 10.1 GPIO and EXTI 10
    - 10.2 UART, USART and LPUART 10
    - 10.3 SPI 11
    - 10.4 I²C 11
    - 10.5 FDCAN 12
    - 10.6 Timers and PWM 13
    - 10.7 ADC 13
    - 10.8 DAC 14
    - 10.9 DMA 14
    - 10.10 Comparators, Op-amps and Math Accelerators 15
    - 10.11 USB Device Firmware 15
    - 10.12 Watchdogs and Fault Handling 15
    - 10.13 Low Power and Wake-up 16
11. RTC and Backup Battery 16
12. Golden Tips Checklist 17
13. Troubleshooting 17
14. References and Revision History 18

1. Introduction

This manual shows how to use the AH_G474 header board and how to get the most out of its STM32G474RBT6. It combines the board facts from the AH_G474 datasheet (Rev V0.1) with MCU tips matched to this board's wiring.

How to read this manual

- **Board facts** (parts, values, pins) come from the AH_G474 datasheet and its schematic excerpts.
- **MCU facts** come from ST's STM32G474 datasheet, RM0440 and AN2606; check their current revisions.
- ★ **Golden tip** boxes are expert tips tied to this board's wiring; **Warning** boxes mark things that can damage it.

What you need

- A USB Type-C data cable (charge-only cables do not enumerate).
- An SWD probe (for example an ST-LINK) with a 3-pin XH cable, or just USB and the BOOT jumper (see 7.4); STM32CubeIDE and STM32CubeProgrammer from ST.

2. Safety and Limits

Limit	Value	Why it matters
Load drawn from the board	**≤ 50 mA**	Higher current damages the 3.3 V regulator (AP2120N-3.3); the MCU and the LEDs share it.
Logic level, MCU supply	**3.3 V**	All header I/O is 3.3 V; read 9.3 before connecting 5 V signals.
Board supply	**5 V, USB Type-C**	The 5 V rail is also on the +5V pins.
PB8 + BOOT jumper	**tied to +3.3 V**	Never drive PB8 low while the jumper is fitted (see 7.2).
CR1220 backup cell	**primary cell**	Not rechargeable: never enable VBAT charging (see 11).

> **Do not draw more than 50 mA from the board.** It damages the regulator. Never connect a supply to the +3.3V pins, or feed the +5V pins while USB is plugged in: no power-path or back-feed protection is documented.

3. Quick Start

1. **Check the BOOT jumper.** For a normal start from Flash leave it open (PB8 is then pulled low by R4).
2. **Power the board.** Plug a USB Type-C data cable into the connector at the left edge. The red POWER LED lights.
3. **Connect the SWD probe.** Wire the probe to the 3-pin XH connector: pin 1 GND, pin 2 SWCLK, pin 3 SWDIO (see 7.3). No probe? Use USB DFU instead (see 7.4).
4. **Create the project.** In STM32CubeIDE choose the part `STM32G474RBTx`. Set *SYS* → *Debug* to *Serial Wire*, configure PC13 as *GPIO_Output* and generate the code. Leave the clock at its default for now.
5. **Blink the LED.** Add the lines below to the main loop, build and run. The green LED on PC13 blinks (PC13 high = LED on).

```
while (1)
{
  HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);   /* PC13: green user LED, high = on */
  HAL_Delay(500);
}
```

> ★ **Golden tip.** First blink at the default clock. After reset the MCU runs from its internal 16 MHz oscillator (HSI16). Get the LED blinking before you configure the PLL: it proves power, SWD and your toolchain. Then move on to the 170 MHz recipe in 6.2.

4. Board Tour

Item	Where	What it is and how to use it
USB Type-C	left edge, top side	5 V supply input and USB 2.0 Full-Speed device port (Section 8).
POWER LED	red, bottom-right edge	On while the board is powered.
PC13 LED	green, top-right edge	User LED: PC13 → R2 (1 kΩ) → LED → GND. PC13 high = on.
NRST	push-button, right of the MCU	Resets the MCU (R1 10 kΩ pull-up, C18 100 nF to GND).
BOOT	jumper, top-left next to USB	Fitted: PB8 = 3.3 V (boot selection, 7.2). Open: PB8 pulled low by R4 (10 kΩ).
SWD	XH 3-pin, right edge	Pin 1 GND, pin 2 SWCLK, pin 3 SWDIO. The same signals are labelled DIO / CLK / GND on the bottom side.
J4 · J3	top and bottom edges	Two 25-pin, 2.54 mm headers with 43 GPIO, +5V, +3.3V and GND (pinout in 9.1).
X1 · X2	next to the MCU	16 MHz crystal (MCU clock) and 32.768 kHz crystal (RTC clock).
CR1220	holder, bottom side	RTC backup battery on VBAT (Section 11).

5. Power

5.1 Power path

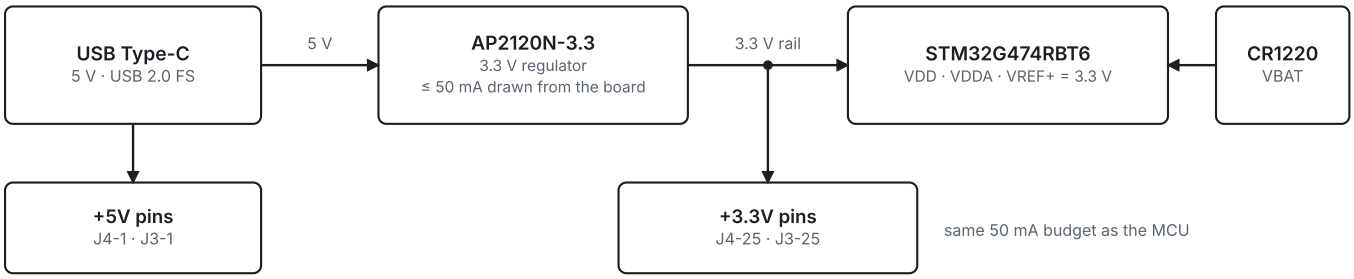


Figure 2. Power path. 5 V arrives through USB Type-C; the AP2120N-3.3 makes the 3.3 V rail; the CR1220 feeds only VBAT.

5 V arrives through the USB Type-C connector and is also brought out to the +5V pins. The AP2120N-3.3 regulator makes the 3.3 V rail for the MCU, the ADC reference and the +3.3V pins. The CR1220 cell feeds only the VBAT pin of the MCU.

5.2 Power budget

The datasheet limit is 50 mA drawn from the board. The MCU, the LEDs and everything you connect to the +3.3V pins share the same regulator, so the current left for your circuits is 50 mA minus what the board itself uses.

Consumer	Typical	Notes
STM32G474RBT6	depends	Set by clock speed and enabled peripherals; it can reach tens of milliamps at 170 MHz (see the IDD tables in the ST datasheet).
PC13 LED	≈ 1 mA	Only when lit: about (3.3 V − 2 V) / 1 kΩ for a green LED.
POWER LED	small	Always on; its resistor value is not given in the datasheet excerpt.
Your circuits	the rest	50 mA minus the lines above.

Heat: an LDO burns the voltage difference, P = (5 V − 3.3 V) × I = 1.7 V × I. At 50 mA that is about 85 mW, all dissipated in the small regulator package.

> ★ **Golden tip.** Measure instead of guessing. Put a USB-C power meter between the PC and the board and read the current with your real firmware running. If it is already near 40 mA, do not add loads; lower SYSCLK or use Sleep mode in idle loops to free budget.

> ★ **Golden tip.** Motors, relays, LED strips and backlit displays need their own supply. Join the grounds (any GND pin on J4 or J3) and never power them from the +3.3V pins.

6. Clocks

6.1 Clock sources

The board carries a 16 MHz crystal X1 (HSE, load capacitors C17 and C19 = 10 pF) for the MCU and a 32.768 kHz crystal X2 (LSE, C13 and C20 = 22 pF) for the RTC. HSI16 (the reset clock), HSI48 (can clock USB with CRS) and LSI (independent watchdog) are inside the MCU.

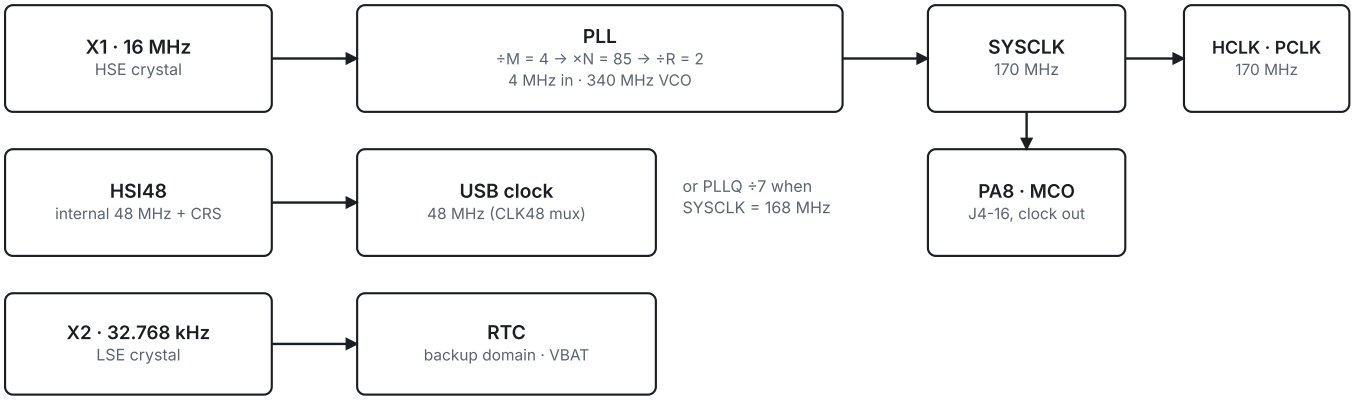


Figure 3. Clock tree for 170 MHz with the 16 MHz crystal. USB runs from HSI48 locked by CRS, or from PLLQ at 168 MHz.

6.2 STM32CubeMX recipe for 170 MHz

Setting	Value
RCC → HSE · LSE	Crystal/Ceramic Resonator for both (LSE only if you use the RTC)
PLL source · M · N · R	HSE · ÷4 · ×85 · ÷2 → SYSCLK = 170 MHz
AHB · APB1 · APB2 prescalers	÷1 · ÷1 · ÷1
Voltage scaling · Flash latency	Range 1 boost mode · 8 wait states (CubeMX sets both at 170 MHz)
USB clock (CLK48)	HSI48, with CRS enabled and SYNC source = USB
Flash	Instruction cache, data cache and prefetch enabled
SYS → Debug	Serial Wire (never “No Debug”)

★ **Golden tip.** USB needs a 48 MHz clock within about ±0.25 %. At 170 MHz the PLL cannot make it from the 16 MHz crystal (340 MHz ÷ 7 = 48.57 MHz, 1.2 % off). Use HSI48 locked by CRS to the USB start-of-frame, or run SYSCLK at 168 MHz (M = 4, N = 84, R = 2) and take PLLQ = ÷7 for exactly 48 MHz.

★ **Golden tip.** Crystal loading. C17 and C19 give X1 roughly 8 to 10 pF once stray capacitance is added; C13 and C20 give X2 roughly 14 to 16 pF. If you swap a crystal, match its specified load capacitance; if the LSE starts slowly, raise the LSE drive level (LSEDRV).

★ **Golden tip.** Check clocks without a scope: route SYSCLK or HSE to MCO on PA8 (J4 pin 16) and measure it. Make sure HSE_VALUE is 16000000 in the HAL configuration, or UART baud rates and timings come out wrong.

★ **Golden tip.** Do not hand-roll the 170 MHz start-up: use the HAL code from CubeMX (voltage scaling, wait states and the intermediate AHB divider follow a fixed sequence). Enable the Clock Security System (CSS) on HSE for products: if the crystal stops, the MCU falls back to HSI16 and raises an NMI.

7. Reset, Boot and Programming

7.1 Reset

The NRST button pulls the reset line to GND; R1 (10 kΩ) holds it high and C18 (100 nF) filters it. The SWD connector has no NRST pin, so a probe cannot reset the board by wire (see 7.3).

7.2 Boot selection (BOOT jumper, PB8)

BOOT jumper	PB8	Boot source with the default option bytes
Open	Low (R4)	Main Flash: your program starts.
Fitted	High (3.3 V)	System memory: the built-in bootloader starts (USB DFU, USART).

PB8 is the BOOT0 pin of the STM32G474RBT6. The boot level is read at reset; the exact boot table depends on the option bytes (nSWBOOT0, nBOOT0, nBOOT1) and is defined in RM0440.

> **Do not drive PB8 low while the BOOT jumper is fitted.** The jumper ties PB8 directly to +3.3 V. Also leave the option bytes nSWBOOT0 and nBOOT0 at their defaults: changing them can make the jumper ineffective.

> ★ **Golden tip.** PB8 is also an I/O (I^2C1_SCL, FDCAN1_RX, TIM16_CH1, TIM4_CH3), but on this board it always carries the 10 kΩ pull-down R4 and, with the jumper fitted, a hard 3.3 V. Avoid PB8 for anything that must be high-impedance at reset. Use PB6 / PB7 for I^2C1 and FDCAN2 (PB5 / PB6) instead.

7.3 SWD connection

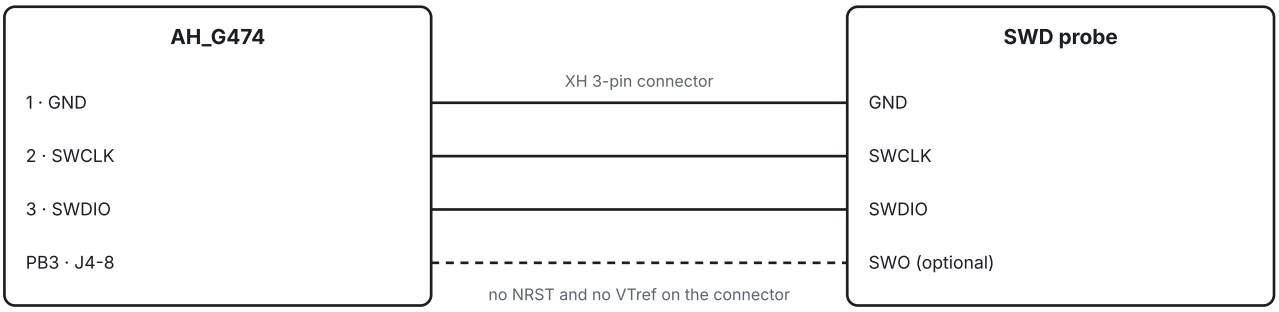


Figure 4. SWD wiring. The connector carries GND, SWCLK and SWDIO; SWO needs an extra wire from PB3.

Pin	Signal	Pad label, bottom side	Probe pin
1	GND	GND	GND
2	SWCLK	CLK	SWCLK / TCK
3	SWDIO	DIO	SWDIO / TMS

> ★ **Golden tip.** SWO printf. The connector has no SWO pin. For ITM / printf trace, wire PB3 (J4 pin 8) to the probe's SWO pin and enable SWO in the debug configuration.

> ★ **Golden tip.** Connect under reset by hand. With no NRST wire, hold the NRST button, click Connect in STM32CubeProgrammer, then release. Use a low SWD clock (1 to 4 MHz) with long leads and keep the GND wire next to SWCLK. In Stop or Standby the debugger can drop unless DBG_STOP / DBG_STANDBY are set in DBGMCU_CR.

7.4 USB DFU (no probe needed)

1. **Fit the BOOT jumper.**
2. **Plug in USB** (or press NRST while it is plugged in). The MCU starts the built-in bootloader.
3. **Open STM32CubeProgrammer,** select USB and click Connect.
4. **Program** your .hex, .bin or .elf (Flash starts at 0x08000000) and start the application.
5. **Remove the jumper** and press NRST to run from Flash.

The ROM bootloader can also be reached over USART1 (PA9 / PA10, J4 pins 15 / 14); see AN2606 for the interfaces of the STM32G47x.

7.5 Recovery and protection

- **Locked out of SWD?** Firmware that remaps the debug pins or sleeps too deeply can block the probe. Fit the BOOT jumper, connect over USB (7.4) and run a full chip erase.
- **Read-out protection:** Level 1 blocks debug access and a regression erases the Flash; Level 2 is permanent. Never set Level 2 on a development board.

8. USB Type-C

- The connector gives 5 V power and a **USB 2.0 Full-Speed device** port. The USB data pins are wired to the MCU and are not on the headers.
- Use a data cable. If a USB-C to USB-C cable from a charger gives no power, try a USB-A to USB-C cable or a PC port.
- In CubeMX enable *USB → Device (FS)* and a class such as CDC (virtual COM port). Set the USB clock as in 6.2. Firmware tips are in 10.11.

> ★ **Golden tip.** Re-enumerate without unplugging. The STM32G4 USB peripheral has a software-controlled internal D+ pull-up. In firmware, call HAL_PCD_DevDisconnect(), wait 100 to 200 ms, then HAL_PCD_DevConnect(): the PC sees an unplug and replug, so you do not have to touch the cable after each download.

9. I/O Headers

9.1 Pinout



Figure 5. Header pinout seen from the top of the board, pin 1 at the left (USB Type-C end). The dot marks the two pins shared with an on-board part.

9.2 Special pins

Pin	Header	Special role	What to know
PB8	J4-3	BOOT0, jumper, R4	See 7.2. Hard 3.3 V when the jumper is fitted.
PC13	J3-4	user LED, R2 1 kΩ	About 1 mA when lit. PC13 sits in the backup domain: treat it as a slow, low-current output (see the pin notes in the ST datasheet).
PA15 · PB4	J4-13 · 7	JTDI · NJTRST	Debug alternate functions at reset (internal pull-ups). They become normal GPIO once firmware sets them, for example with Debug = Serial Wire.
PB3	J4-8	JTDO / SWO	Use it for SWO trace (7.3) or as GPIO.
PA8	J4-16	MCO	Clock output for measurements (6.2).
PA4 · PA5	J3-13 · 14	DAC1 outputs	Analog outputs (see 10.8).

9.3 Electrical tips

- **3.3 V logic.** The board adds no level shifting. Many (not all) STM32G4 pins are 5 V tolerant as digital inputs, but not in analog mode: check the pin type (FT, FT_a ...) in the ST datasheet, or use a level shifter or series resistor.
- **Unused pins:** set them to analog mode to cut leakage and noise.
- **Drive current:** the datasheet limits current per pin and for the whole package. Use transistors or driver ICs for motors, relays and LED strips.
- **Header lines go straight to the MCU** (no series resistors or ESD parts in the schematic excerpt). Protect long cables with a series resistor and a TVS or clamp.

> ★ **Golden tip.** Plan pins before wiring. Many functions share pins (PA9 is USART1_TX, I²C2_SCL and TIM1_CH2 at once). Pick peripherals in CubeMX first, let it resolve conflicts, then print the pin list and wire from it. Prefer FDCAN2 (PB5 / PB6) and I²C1 (PB6 / PB7) to stay away from the BOOT pin PB8.

9.4 Peripheral routing guide

Peripheral	Signals → header pins	Notes
USART1	TX PA9 (J4-15) · RX PA10 (J4-14)	Alternatives: PB6 / PB7, PC4 / PC5.
USART2	TX PA2 (J3-11) · RX PA3 (J3-12)	
USART3	TX PB10 (J3-22) · RX PB11 (J3-23)	Alternative: PC10 / PC11.
UART4 · UART5	UART4 TX PC10 (J4-12) · RX PC11 (J4-11) UART5 TX PC12 (J4-10) · RX PD2 (J4-9)	UART4 shares pins with SPI3 and USART3.
LPUART1	TX PC1 (J3-6) · RX PC0 (J3-5)	Low-power UART.
I²C1	SCL PB6 (J4-5) · SDA PB7 (J4-4)	PB8 / PB9 also work, but PB8 is the BOOT pin.
I²C2 · I²C3	I²C2 SCL PA9 (J4-15) · SDA PA8 (J4-16) I²C3 SCL PC8 (J4-18) · SDA PC9 (J4-17)	I²C needs external pull-ups (none documented on the headers).
SPI1	SCK PA5 (J3-14) · MISO PA6 (J3-15) · MOSI PA7 (J3-16) · NSS PA4 (J3-13)	
SPI2	SCK PB13 (J4-23) · MISO PB14 (J4-22) · MOSI PB15 (J4-21) · NSS PB12 (J4-24)	
SPI3	SCK PC10 (J4-12) · MISO PC11 (J4-11) · MOSI PC12 (J4-10) · NSS PA15 (J4-13)	
FDCAN	FDCAN2 RX PB5 (J4-6) · TX PB6 (J4-5) (recommended) FDCAN1 RX PB8 (J4-3) · TX PB9 (J3-3) FDCAN3 RX PB3 (J4-8) · TX PB4 (J4-7)	Needs an external CAN transceiver. FDCAN1 uses the BOOT pin PB8.
ADC inputs	PA0 (J3-9) … PA7 (J3-16); PC0 (J3-5) … PC5 (J3-18); PB0 (J3-19) … PB2 (J3-21); PB11 … PB15	Analog mode is not 5 V tolerant.
DAC outputs	DAC1_OUT1 PA4 (J3-13) · DAC1_OUT2 PA5 (J3-14) · DAC2_OUT1 PA6 (J3-15)	
TIM1 (advanced)	CH1–CH3 PA8 (J4-16), PA9 (J4-15), PA10 (J4-14) CH1N–CH3N PA7 (J3-16), PB0 (J3-19), PB1 (J3-20) · BKIN PA6 (J3-15)	Three-phase PWM with dead time and break input.
TIM8 (advanced)	CH1–CH4 PC6 (J4-20), PC7 (J4-19), PC8 (J4-18), PC9 (J4-17)	
HRTIM1	A1/A2 PA8/PA9 · B1 PA10 · C1/C2 PB12/PB13 · D1/D2 PB14/PB15 · E1/E2 PC8/PC9 · F1/F2 PC6/PC7	High-resolution timer outputs of the STM32G474.
TIM2 · TIM3 · TIM4	TIM2 CH1–CH4 PA0–PA3 · TIM3 CH1–CH4 PA6, PA7, PB0, PB1 · TIM4 CH1–CH4 PB6–PB9	General-purpose timers (TIM2 is 32-bit).
Debug · clock out	SWO PB3 (J4-8) · MCO PA8 (J4-16)	SWO needs a wire to the probe (see 7.3).
Wake-up pins	WKUP1 PA0 (J3-9) · WKUP2 PC13 (J3-4) · WKUP4 PA2 (J3-11) · WKUP5 PC5 (J3-18)	Wake from Standby / Shutdown.

Selected, commonly used mappings. Always confirm them in STM32CubeMX or in the alternate-function table of the STM32G474 datasheet. Per-peripheral tips are in Section 10.

10. Peripheral Guide

Peripheral-by-peripheral tips for the STM32G474RBT6 on this board. Each part lists the pins on the headers, the setup that works and the traps that cost the most time. Code uses the STM32 HAL; handle names come from CubeMX. The pin lists repeat the routing guide in 9.4: confirm them in CubeMX before you wire anything.

10.1 GPIO and EXTI

- **Clock first.** Enable the port clock before touching a pin (CubeMX does it; with registers use RCC_AHB2ENR).
- **Reset state.** At reset every pin is analog except the debug pins (PA13, PA14, PA15, PB3, PB4). Unused pins draw no current and floating inputs cannot oscillate.
- **Speed.** Use Low for LEDs and slow signals; High or Very high only for fast buses such as SPI above 10 MHz. Faster edges mean more EMI and overshoot.
- **Atomic writes.** HAL_GPIO_WritePin uses BSRR and is safe against interrupts; a read-modify-write of ODR is not.
- **Defined input levels.** Use the internal pull-up or pull-down (tens of kΩ) or an external resistor: a floating input picks up noise.

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)      /* one callback for all EXTI lines */
{
  if (pin == GPIO_PIN_0) { /* PA0, PB0 or PC0: whichever port owns line 0 */ }
}
```

> ★ **Golden tip.** EXTI lines are shared by pin number. Line n serves pin n of exactly one port, so PA0 (J3-9), PB0 (J3-19) and PC0 (J3-5) cannot all be interrupt inputs; the same holds for PA8 / PB8 / PC8 and every other number. Lines 5 to 9 share one vector (EXTI9_5_IRQn) and lines 10 to 15 share EXTI15_10_IRQn, so test the pin inside the callback. Debounce buttons with 10 to 20 ms of software or an RC filter, and keep the ISR to setting a flag.

10.2 UART, USART and LPUART

Pins on the board. USART1 TX PA9 (J4-15), RX PA10 (J4-14) · USART2 TX PA2 (J3-11), RX PA3 (J3-12) · USART3 TX PB10 (J3-22), RX PB11 (J3-23) · UART4 TX PC10 (J4-12), RX PC11 (J4-11) · UART5 TX PC12 (J4-10), RX PD2 (J4-9) · LPUART1 TX PC1 (J3-6), RX PC0 (J3-5)

- **Wiring.** 3.3 V TTL, TX to RX crossed, common GND. Use a 3.3 V USB-serial adapter, never RS-232 levels.
- **Kernel clock.** Each USART can run from PCLK, SYSCLK, HSI16 or LSE. HSI16 keeps the baud rate fixed even if you change the PLL later; LPUART1 on the LSE (X2) gives 9600 baud and wake-up from Stop.
- **Baud accuracy.** With 16× oversampling BRR = f_{clk} / baud, so the table below applies at 170 MHz. Keep the sum of both ends' errors under about 2 %. The maximum is 10.6 Mbit/s (21.25 Mbit/s with 8× oversampling).
- **Receive.** Use circular DMA plus the idle-line interrupt for variable-length frames. On an overrun error re-arm the receiver in HAL_UART_ErrorCallback.

Baud rate	9600	115200	460800	921600	1 M	2 M	5 M
BRR (÷16)	17708	1476	369	184	170	85	34
Error	+0.002 %	−0.02 %	−0.02 %	+0.25 %	0 %	0 %	0 %

```
int __io_putchar(int ch)                                /* printf over USART1 */
{ HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 10); return ch; }

HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rx_buf, sizeof rx_buf);   /* variable-length frames */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *h, uint16_t size) { /* size bytes arrived */ }
```

> ★ **Golden tip.** RS-485 without a direction pin. The USART has a hardware driver-enable output: USART2_DE on PA1 (J3-10) and USART3_DE on PB14 (J4-22) (pin function RTS_DE). Set the assertion and de-assertion times in the USART settings so the transceiver is enabled before the first bit and released after the last.

> ★ **Golden tip.** printf with floats prints nothing in newlib-nano unless the linker option "Use float with printf" (-u _printf_float) is enabled. PA9 and PA10 also carry I²C2_SCL and TIM1 channels, so pick one function per pin.

10.3 SPI

Pins on the board. SPI1 SCK PA5 (J3-14), MISO PA6 (J3-15), MOSI PA7 (J3-16), NSS PA4 (J3-13) · SPI2 SCK PB13 (J4-23), MISO PB14 (J4-22), MOSI PB15 (J4-21), NSS PB12 (J4-24) · SPI3 SCK PC10 (J4-12), MISO PC11 (J4-11), MOSI PC12 (J4-10), NSS PA15 (J4-13)

- **Software chip select.** In CubeMX set NSS = Software and drive each slave's CS from an ordinary GPIO output. It is simpler and works with several slaves.
- **Mode.** Match CPOL and CPHA to the slave's datasheet (mode 0 is the most common). A wrong mode shows up as shifted or garbled bits.
- **Speed.** Start at 10 MHz or less, then raise it with the pin speed set to High or Very high. Keep SCK, MOSI and MISO short with a common GND; add 22 to 47 Ω in series at the master end for fast clocks.
- **DMA.** For long transfers use HAL_SPI_TransmitReceive_DMA and release CS only in the transfer-complete callback.
- **Register-level code.** For 8-bit frames set FRXTH (RXNE threshold) and access DR as 8 bits, otherwise RXNE waits for 16 bits.

Prescaler	÷2	÷4	÷8	÷16	÷32	÷64	÷128	÷256
SCK at 170 MHz (MHz)	85	42.5	21.25	10.63	5.31	2.66	1.33	0.66

```
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);
HAL_SPI_TransmitReceive(&hspi1, tx, rx, len, 10);
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);       /* CS idles high */
```

> ★ **Golden tip.** Every pin starts analog at reset, so CS lines float while the MCU boots and a slave may see random clocks. Fit a 10 kΩ pull-up on each CS line. SPI1's NSS and SCK pins (PA4, PA5) are also the DAC1 outputs: if you need both, move SPI1 to PB3 / PB4 / PB5 or use SPI2.

10.4 I²C

Pins on the board. I²C1 SCL PB6 (J4-5), SDA PB7 (J4-4) · I²C2 SCL PA9 (J4-15), SDA PA8 (J4-16) · I²C3 SCL PC8 (J4-18), SDA PC9 (J4-17)

- **Pull-ups.** I²C lines are open-drain: they need external pull-ups to +3.3V. The datasheet excerpt shows none on the header pins, so add them or use a sensor module that has them.
- **Timing.** Let CubeMX compute TIMINGR for the real kernel clock. If you change SYSCLK, PCLK1 or the I²C clock source later, regenerate it.
- **Address.** HAL wants the 8-bit form: 7-bit address shifted left by one (0x68 becomes 0xD0).
- **Stuck bus.** If SDA is held low after a reset in the middle of a transfer, switch SCL to open-drain GPIO, pulse it up to 9 times, send a STOP, then re-initialise the peripheral.
- **3.3 V only.** Use a level shifter for 5 V devices.

Mode	SCL	Max rise time	Typical pull-up
Standard	100 kHz	1000 ns	4.7 kΩ
Fast	400 kHz	300 ns	2.2 to 3.3 kΩ
Fast-mode Plus	1 MHz	120 ns	1 to 1.5 kΩ, with Fm+ drive enabled (HAL_I2CEx_EnableFastModePlus)

```
for (uint8_t a = 1; a < 127; a++)                              /* I2C scanner */
  if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 2, 5) == HAL_OK) printf("found 0x%02X\r\n", a);
```

★ **Golden tip.** Size the pull-ups from the bus capacitance: $R \le t_r / (0.85 \times C_{bus})$ and $R \ge (3.3\ V - 0.4\ V) / 3\ mA \approx 1\ k\Omega$. Modules often carry their own pull-ups (commonly 4.7 or 10 kΩ) and they add in parallel: three modules with 10 kΩ each leave only 3.3 kΩ on the bus. Prefer I²C1 on PB6 / PB7: PB8 / PB9 would put the BOOT jumper on the clock line.

10.5 FDCAN

Pins on the board. FDCAN2 RX PB5 (J4-6), TX PB6 (J4-5) (recommended) · FDCAN1 RX PB8 (J4-3), TX PB9 (J3-3) · FDCAN3 RX PB3 (J4-8), TX PB4 (J4-7)

- **Transceiver.** The MCU pins are logic-level: a CAN transceiver is required and the board has none. For CAN FD pick a transceiver rated for it (for example TJA1051T/3 or MCP2562FD, both usable with 3.3 V logic).
- **Bus.** 120 Ω termination at the two ends of the bus only, twisted pair, and a common ground reference between nodes.
- **Filters.** After reset the peripheral accepts no frames: configure a filter or HAL_FDCAN_ConfigGlobalFilter to accept frames into FIFO 0.
- **Bench test.** Internal loopback mode (FDCAN_MODE_INTERNAL_LOOPBACK) exercises the peripheral with no transceiver at all.
- **Same settings everywhere.** All nodes must agree on classic CAN versus FD, bit rate and sample point.

Bit rate (16 MHz kernel clock)	Prescaler	tq per bit	Seg1	Seg2	Sample point
1 Mbit/s	1	16	13	2	87.5 %
500 kbit/s	2	16	13	2	87.5 %
250 kbit/s	4	16	13	2	87.5 %
125 kbit/s	8	16	13	2	87.5 %
FD data phase, 2 Mbit/s	1	8	6	1	87.5 %

```
HAL_FDCAN_ConfigGlobalFilter(&hfdcan2, FDCAN_ACCEPT_IN_RX_FIFO0, FDCAN_ACCEPT_IN_RX_FIFO0,
                             FDCAN_REJECT_REMOTE, FDCAN_REJECT_REMOTE);
HAL_FDCAN_Start(&hfdcan2);
HAL_FDCAN_ActivateNotification(&hfdcan2, FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);
```

★ **Golden tip.** Use the 16 MHz crystal as the FDCAN kernel clock (RCC → FDCAN clock source = HSE). A crystal is far more accurate than the PLL-derived or RC clocks, and 16 MHz divides cleanly into 16 time quanta per bit for every standard rate (table above). Sync is one tq and Seg1 includes the propagation segment; use SJW of 1 to 2.

10.6 Timers and PWM

Pins on the board. TIM1 CH1–CH3 PA8 (J4-16), PA9 (J4-15), PA10 (J4-14) · CH1N–CH3N PA7 (J3-16), PB0 (J3-19), PB1 (J3-20) · BKIN PA6 (J3-15) · TIM8 CH1–CH4 PC6 (J4-20), PC7 (J4-19), PC8 (J4-18), PC9 (J4-17) · TIM2 CH1–CH4 PA0–PA3 · TIM3 PA6, PA7, PB0, PB1 · TIM4 PB6–PB9

- **Timer clock.** With the APB prescalers at ÷1 (6.2) the timers run at 170 MHz. With a prescaler of ÷2 or more the timer clock is twice PCLK.
- **PWM maths.** $f_{PWM} = f_{timer} / ((PSC + 1) \times (ARR + 1))$; resolution is about $\log_2(ARR + 1)$ bits. Centre-aligned mode halves the frequency for the same ARR.
- **Preload.** Keep ARR and CCR preload enabled so changes take effect cleanly at the update event and the output never glitches.
- **Servo example.** PSC = 169 (1 MHz tick), ARR = 19999 (50 Hz), CCR = 1000 to 2000 for 1 to 2 ms pulses.
- **Encoder.** TIM3 (PA6 / PA7) or TIM4 (PB6 / PB7) in encoder mode with the input filter on; open-collector encoders need pull-ups (the internal ones are weak).
- **Input capture.** TIM2 is 32 bits wide: use it for long periods and for PWM-input measurements on PA0 to PA3.

PWM frequency	20 kHz	50 kHz	100 kHz	500 kHz	1 MHz
Counts per period (ARR + 1)	8500	3400	1700	340	170
Resolution	≈ 13.1 bit	≈ 11.7 bit	≈ 10.7 bit	≈ 8.4 bit	≈ 7.4 bit

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);          /* PA8  */
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1);       /* PA7, complementary with dead time */
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, (__HAL_TIM_GET_AUTORELOAD(&htim1) + 1) / 2);   /* 50 % */
```

★ **Golden tip.** Advanced timers (TIM1, TIM8) for power stages. One dead-time tick is about 5.9 ns at 170 MHz; choose the dead time from the gate-driver and MOSFET datasheets (often 100 to 500 ns). Configure the break input (BKIN on PA6, J3-15) and the idle output states before enabling the outputs, start both the PWM and the PWMN channel, and keep MOE set only while the power stage is meant to run. TIM1 pins PA7 / PB0 / PB1 are also ADC and SPI pins: plan them first.

★ **Golden tip.** Microsecond timing without a timer: enable the Cortex-M4 cycle counter. At 170 MHz, 170 cycles are exactly 1 µs. HRTIM1 (STM32G474) goes much further, with 184 ps resolution on PA8 to PA10, PB12 to PB15 and PC6 to PC9: use the CubeMX HRTIM wizard for digital power conversion.

```
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;   /* enable the DWT unit */
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;              /* DWT->CYCCNT counts CPU cycles */
```

10.7 ADC

Pins on the board. PA0 (J3-9) … PA7 (J3-16) · PC0 (J3-5) … PC5 (J3-18) · PB0 (J3-19) … PB2 (J3-21) · PB11 … PB15. Internal channels: temperature sensor, VBAT/3 and VREFINT.

- **Clock.** The ADC clock must stay at or below 60 MHz. In CubeMX use the asynchronous clock from SYSCLK ÷4 (42.5 MHz).
- **Conversion time.** For 12 bits, t_{conv} = (sampling time + 12.5) ADC cycles. The table below uses a 42.5 MHz ADC clock.

- **Sampling time.** Pick it from the source resistance using the datasheet table: a high-impedance source needs 92.5 cycles or more; buffer it with an op-amp or add a capacitor at the pin.
- **Calibrate** after every power-up, before the first conversion.
- **Oversampling.** The hardware oversampler (up to 256×, with a right shift) gives up to 16-bit results and averages noise without CPU work.
- **Never exceed VREF+.** Analog pins must not see more than 3.3 V; scale larger signals with a divider and mind its source resistance.

Sampling time (cycles)	2.5	6.5	12.5	24.5	47.5	92.5	247.5	640.5
Conversion time (µs)	0.35	0.45	0.59	0.87	1.41	2.47	6.12	15.4

```
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);        /* once after power-up */
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buf, N);            /* circular DMA, half/full callbacks */
HAL_TIM_Base_Start(&htim6);                                   /* TIM6 TRGO triggers every conversion */
```

★ **Golden tip.** Sample on a timer, not in a loop. Trigger the ADC from a timer TRGO (TIM6 for plain sampling, an advanced timer for motor control) and move the data with circular DMA: the sample rate is then exact and independent of the CPU. For current sensing use injected channels triggered at the centre of the PWM period, away from switching edges. Read the temperature sensor and VBAT with long sampling times (several microseconds) and convert with the factory calibration values (LL macros __LL_ADC_CALC_TEMPERATURE and __LL_ADC_CALC_VREFANALOG_VOLTAGE).

10.8 DAC

Pins on the board. DAC1_OUT1 PA4 (J3-13) · DAC1_OUT2 PA5 (J3-14) · DAC2_OUT1 PA6 (J3-15)

- **Output buffer.** With the buffer on, the output drives small resistive loads but swings roughly from 0.2 V to VDDA − 0.2 V (see the datasheet for the exact limits). With it off the output is a high-impedance node: buffer it externally.
- **Waveforms.** Fill a lookup table, run the DAC from a timer trigger (TIM6 or TIM7) and let circular DMA feed it. The sample rate is the timer update rate; the waveform frequency is the sample rate divided by the table length.
- **Reference.** The DAC reference is VREF+ = 3.3 V from the LDO, so absolute accuracy is only as good as that rail; drive ratiometric circuits from the same 3.3 V.
- **Offset trim.** Run HAL_DACEx_SelfCalibrate at start-up when the output buffer is used.
- **Filtering.** For audio or smooth analog levels add an RC low-pass after the pin.

```
HAL_DAC_Start_DMA(&hdac1, DAC_CHANNEL_1, (uint32_t *)sine, 64, DAC_ALIGN_12B_R);
HAL_TIM_Base_Start(&htim6);                    /* 64 samples per period: f_wave = f_TIM6 / 64 */
```

★ **Golden tip.** A 1 kHz sine from 64 samples needs a 64 kHz trigger: with TIM6 on the 170 MHz clock use PSC = 0 and ARR = 2655 (170 MHz / 2656 = 64.01 kHz). Pins PA4 and PA5 double as SPI1 NSS / SCK, and PA6 as TIM1_BKIN and SPI1_MISO, so decide early which function owns them.

10.9 DMA

- **Init order.** The DMA clock must be on before the peripheral that uses it: MX_DMA_Init() has to run before MX_USARTx_UART_Init(), MX_ADCx_Init() and similar calls. Wrong order is the classic cause of “DMA never starts”.
- **Data width.** Memory and peripheral widths must match the data: bytes for UART and 8-bit SPI, half-words for 12-bit ADC and DAC data. A mismatch gives garbage that looks like noise.

- **Circular mode with callbacks.** Use half-transfer and full-transfer callbacks as a double buffer: process one half while the DMA fills the other.
- **Routing and priorities.** The STM32G474 has two DMA controllers with 8 channels each and a DMAMUX that connects any request to any channel: CubeMX assigns them. Give time-critical streams (ADC, DAC) the highest priority.
- **Buffers.** Keep DMA buffers in the normal SRAM defined by the linker script, make them global or static (never on a stack that goes away) and do not touch a buffer while the DMA owns it.

10.10 Comparators, Op-amps and Math Accelerators

- **Comparators.** The STM32G474 has seven fast comparators. Their outputs can drive a timer break input in hardware, giving an over-current trip in nanoseconds with no CPU involved; an internal DAC channel can supply the threshold.
- **Op-amps.** Six on-chip op-amps can run as followers or in programmable-gain mode (gain up to 64) to amplify a shunt signal straight into the ADC. Their pins are shared with ADC and GPIO pins, so check the pin-definition table for the 64-pin package before you commit to a layout.
- **CORDIC.** A hardware unit for sine, cosine, arctangent and square root: useful for Park and Clarke transforms in motor control. Data is fixed-point (q1.31 or q1.15) and angles are scaled by 1/π.
- **FMAC.** A filter engine for FIR and IIR filters fed by DMA, which offloads the core in signal-processing loops.

10.11 USB Device Firmware

- **Class.** CDC (virtual COM port) is the easiest way to get logs and commands over the Type-C cable. HID suits keyboards and mice, DFU suits field updates and MSC suits storage.
- **CDC transmit.** CDC_Transmit_FS returns USBD_BUSY while the previous packet is still going: use a ring buffer and the transmit-complete callback instead of calling it in a tight loop.
- **First message lost?** Data sent before the PC opens the port is dropped: wait for the host's DTR line state before the first print.
- **Baud rate.** The rate a terminal sets on a virtual COM port is ignored; the link always runs at USB speed.
- **VID / PID.** 0x0483 / 0x5740 are ST's demo values for a virtual COM port. They are fine for tests, but a product needs its own VID and PID.
- **Memory.** If the device does not enumerate, check the heap and stack sizes in the linker script: the USB middleware allocates memory.

> ★ **Golden tip.** Clock and re-enumeration decide whether USB feels solid. Use HSI48 locked by CRS (or the 168 MHz option) as described in 6.2, and use HAL_PCD_DevDisconnect / HAL_PCD_DevConnect at start-up so the PC always sees a fresh connection after a download.

10.12 Watchdogs and Fault Handling

- **Independent watchdog (IWDG).** It runs from the LSI. With the prescaler at ÷32 and the reload value at 999 the timeout is about one second. The LSI tolerance is wide, so use the IWDG to catch lock-ups, not to measure time.
- **Window watchdog (WWDG).** It runs from the APB clock and also resets the MCU when you refresh it too early, which catches runaway loops.
- **Freeze while debugging.** Call __HAL_DBGMCU_FREEZE_IWDG() in debug builds, otherwise the watchdog resets the MCU every time you stop at a breakpoint.
- **HardFault.** Do not leave the default endless loop. Store the stacked PC, then blink the PC13 LED in a pattern or print it over SWO: a visible fault saves hours.

- **Brown-out and reset cause.** Set the brown-out reset level that matches your supply, and read the reset flags (RCC_CSR) at start-up to see whether the last reset was watchdog, brown-out or the NRST button.

10.13 Low Power and Wake-up

- **Modes.** Sleep stops the CPU only; Stop 0 and Stop 1 keep the RAM and wake in microseconds; Standby keeps only the backup domain and wakes through a reset; Shutdown is the lowest-power mode.
- **Wake-up sources.** Wake-up pins PA0 (J3-9), PC13 (J3-4), PA2 (J3-11) and PC5 (J3-18), the RTC alarm or wake-up timer (clocked by the LSE crystal X2), and LPUART1 on the LSE.
- **After Standby** the program starts from reset. Check the standby flag (PWR_FLAG_SB), clear it, and restore your state from the backup registers.
- **Pin states.** In Standby the pins float. To hold a level use the PWR pull-up and pull-down configuration registers (enable them with the APC bit).
- **Debugging.** The debug link can drop in Stop and Standby: set DBG_STOP and DBG_STANDBY in DBGMCU_CR while developing.

```
if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB)) { __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB); /* woke from Standby */ }
```

> ★ **Golden tip.** Do not benchmark micro-amps on this board. The POWER LED, the LDO's own quiescent current and the PC13 LED all draw more than a sleeping STM32G4, and no IDD measurement jumper is documented. Judge the MCU's low-power numbers from the ST datasheet, or from the VBAT current with USB removed; judge the board's battery life by measuring the whole board.

11. RTC and Backup Battery

The 32.768 kHz crystal X2 clocks the RTC and the CR1220 holder on the bottom side feeds VBAT, so the RTC and the backup registers keep running when USB is removed.

1. **Fit the cell** in the holder (observe the polarity marking).
2. **Enable backup-domain access,** start the LSE and select it as RTC clock (CubeMX: RCC → LSE crystal, RTC clock source LSE).
3. **Set the time once** and write a "magic" value to a backup register (HAL_RTCEx_BKUPWrite). At the next boot read it: if it matches, the RTC is already running and must not be re-initialised.

> **Never enable VBAT charging with the CR1220.** The STM32G4 can charge VBAT through an internal resistor (PWR_CR4 VBE bit). A CR1220 is a primary lithium cell: keep VBE at 0.

> ★ **Golden tip.** Watch the cell. Read the internal VBAT/3 ADC channel (HAL: ADC_CHANNEL_VBAT) and warn the user when it drops under about 2.5 V. Estimate cell life as capacity divided by the backup-domain current given in the ST datasheet.

> ★ **Golden tip.** Time accuracy and API traps. A typical 32.768 kHz crystal is good to about ±20 ppm, which is $20 \times 10^{-6} \times 86\,400$ s ≈ 1.7 s per day; trim it with the smooth calibration (HAL_RTCEx_SetSmoothCalib). Always call HAL_RTC_GetTime before HAL_RTC_GetDate: reading the time locks the shadow registers and reading the date unlocks them. For periodic wake-ups use HAL_RTCEx_SetWakeUpTimer_IT.

12. Golden Tips Checklist

Before first power-up

- BOOT jumper open
- Nothing connected to the +5V / +3.3V pins as a supply
- External load planned under 50 mA in total
- CR1220 polarity checked

Before you program

- GND, SWCLK, SWDIO wired (SWO on PB3 if needed)
- Debug = Serial Wire; HSE_VALUE = 16000000
- USB clock from HSI48 + CRS
- Unused pins in analog mode; VBAT charging off

Before you ship

- Real firmware current measured against 50 mA
- CSS and watchdog enabled
- RDP decision made (never Level 2 on dev boards)
- USB re-enumeration tested

Peripheral pre-flight

Peripheral	Check before you test
GPIO · EXTI	Port clock on; one port per EXTI line; pull-ups or pull-downs on inputs; PB8 and PC13 limits respected.
UART	TX and RX crossed, common GND, 3.3 V adapter, baud error below 1 % (10.2).
SPI	Mode and speed match the slave; CS idles high with a 10 kΩ pull-up; DMA transfers finish before CS rises.
I²C	External pull-ups present; 7-bit address shifted left; TIMINGR regenerated after clock changes.
FDCAN	Transceiver fitted; 120 Ω at both bus ends; filter configured; HSE as kernel clock; same bit timing on all nodes.
PWM · TIM1	Dead time and break configured; outputs safe at reset; PWM and PWMN both started; MOE set only when intended.
ADC	Calibrated; sampling time fits the source impedance; input never above 3.3 V; timer trigger and DMA set up.
DAC	Buffer setting matches the load; trigger timer running; table length and sample rate give the wanted frequency.
DMA	MX_DMA_Init before peripheral init; data widths match; buffers global or static.
USB	HSI48 + CRS or the 168 MHz option; data cable; re-enumeration tested; heap and stack large enough.

13. Troubleshooting

Symptom	Likely cause	What to do
POWER LED is off	Bad cable or port, short on a header	Try another cable and port; unplug everything from the headers; look for +5V / +3.3V shorted to GND.
PC does not see the USB device	Charge-only cable; USB clock not set; firmware not running	Use a data cable; set HSI48 + CRS (6.2); try DFU with the BOOT jumper (7.4).
SWD: “no target”	Wiring, firmware disabled SWD, low-power mode	Check the three wires and GND; lower the SWD clock; connect under reset with the NRST button; erase through DFU (7.4).
Board enters the bootloader at every reset	BOOT jumper fitted, or PB8 pulled high	Remove the jumper; check external wiring on PB8.
Green LED does not blink	Wrong pin, clock or polarity	PC13 high = on; enable the GPIOC clock; set PC13 as output.
Baud rates or timings are wrong	HSE_VALUE or PLL mismatch	Set HSE_VALUE = 16000000; verify SYSCLK on MCO (PA8).

System hangs in SystemClock_Config	HSE not starting or bad PLL values	Compare with 6.2; check X1 and its 10 pF capacitors; keep an HSI16 fallback while testing.
UART prints garbage	Baud or clock mismatch, TX / RX swapped, 5 V or RS-232 adapter	Check the table in 10.2; swap TX and RX; verify SYSCLK on MCO; use a 3.3 V adapter.
I²C device not found	No pull-ups, address not shifted, wrong pins, stale TIMINGR	Add 2.2 to 4.7 kΩ pull-ups; use address << 1; run the scanner (10.4).
SPI returns 0xFF or 0x00	Wrong mode, CS not asserted, MISO unconnected, slave unpowered	Check CPOL / CPHA and CS; scope SCK; add a CS pull-up (10.3).
TIM1 PWM output missing	MOE or break active, wrong pin function, complementary channel not started	Start both PWM and PWMN; check BKIN; verify the alternate function in CubeMX (10.6).
CAN has no activity or errors	No transceiver or termination, wrong bit timing, filter rejects frames	Check 10.5: transceiver, 120 Ω, 16 tq timing, global filter; try internal loopback.
DMA transfers give garbage	Data width mismatch, init order, buffer overwritten	Match widths; call MX_DMA_Init() first; use circular mode with half / full callbacks (10.9).
ADC reads 0 or 4095	Pin not in analog mode, wrong channel, input above 3.3 V	Configure the pin as analog; check the channel; limit the input to VREF+ (10.7).
ADC readings jump or are noisy	Reference noise, source impedance	Calibrate; lengthen sampling; oversample; trigger from a timer; use VREFINT (10.7).
RTC loses the time at power-off	No cell, LSE not started, backup domain reset	Fit a CR1220; start the LSE; use the backup-register check (Section 11).
Board is hot or resets	Load above 50 mA	Remove loads; measure the current; use a separate supply (Section 5).
MCU resets while debugging	IWDG still running at a breakpoint	Call __HAL_DBGMCU_FREEZE_IWDG() in debug builds (10.12).

14. References and Revision History

- AH_G474 Datasheet, Rev V0.1 (this board).
- STM32G474xB/C/E datasheet and RM0440 (STM32G4 series reference manual), ST.
- AN2606: STM32 microcontroller system memory boot mode, ST.
- PM0214: STM32 Cortex-M4 programming manual, ST.
- STM32CubeIDE, STM32CubeMX and STM32CubeProgrammer user documentation, and the STM32G4 HAL and LL driver description, ST.

Doc rev	Date	Hardware	Changes
V0.1	November 2026	V0.1	Initial release.
V0.2	November 2026	V0.1	Added the peripheral guide (Section 10) with tables and code, RTC tips, a peripheral pre-flight checklist and more troubleshooting entries.

@AliasgharZ01 — Board facts in this manual come from the AH_G474 datasheet (Rev V0.1). MCU facts and tips refer to ST's documents and should be checked against their current revisions. Drawing more than 50 mA from the board damages the voltage regulator. The user is responsible for the safety of the external circuits connected to the board.